



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS PROGRAMA
DE INGENIERÍA DE SISTEMAS



GUÍA PRÁCTICA SEMANA 07

Asignatura: Desarrollo de Aplicaciones Web (IS093A)

Unidad I: Desarrollo Web Frontend

Tema: React Hooks: useState, useEffect, useContext, useRef, useReducer, useCallback, useMemo y Hooks Personalizados

Modalidad: Laboratorio

INTEGRANTES:

- Barja Ortiz Erick Gerson
- Navarro Serva Lesly Brenda
- Toribio Anselmo David Angel
- Yauri Torres Benjamin Raul

OBJETIVO DE LA PRÁCTICA

Deben desarrollar un **Dashboard de Gestión con Estado Complejo** que cumpla:

1. [useReducer](#) para estado de lista (add, remove, filter, sort) + [useState](#) para UI local.
2. [useContext](#) para tema claro/oscuro o preferencias de usuario.
3. [useEffect](#) para sincronizar preferencias con [localStorage](#) y cleanup.
4. [useMemo](#) para lista filtrada/ordenada, [useCallback](#) para handlers de botones.
5. [useRef](#) para auto-focus en input o acceso a nodo DOM sin re-render.
6. Hook personalizado [useLocalStorage](#) o [useDebounce](#) reutilizable.

Roles Técnicos

Rol	Responsabilidad
Arquitecto de Estado	useReducer , acciones, reducer function, inmutabilidad
Ingeniero de Efectos & Contexto	useEffect , useContext , sync con storage, cleanup
Optimizador de Rendimiento	useMemo , useCallback , useRef , análisis Profiler
QA & Hook Validator	Reglas de Hooks, ESLint warnings, documentación técnica, custom hook



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS PROGRAMA
DE INGENIERÍA DE SISTEMAS



Entregable

- Repositorio GitHub o ZIP con: [package.json](#), [src/](#), [README.md](#)
- [README.md](#) debe incluir:
 - o Diagrama de flujo de estado y ciclo de render
 - o Justificación técnica de cada hook utilizado (¿por qué no useState simple?)
 - o Capturas de React DevTools Profiler (commits, tiempo de render, componentes optimizados)

Rúbrica de Evaluación

Criterio	Peso
Uso correcto de useReducer + manejo de estado complejo	25%
useContext + useEffect (sync, cleanup, dependencias)	20%
useMemo / useCallback aplicados estratégicamente (sin overuse)	20%
useRef + Hook personalizado funcional y reutilizable	20%
Validación Profiler/ESLint, documentación y trabajo en equipo	15%



CÓDIGO BASE DE LA GUÍA PRÁCTICA (Refactorizar) src/contexts/ThemeContext.jsx

jsx

```
1 import { createContext, useState, useEffect } from 'react'
2
3 export const ThemeContext = createContext()
4
5 export const ThemeProvider = ({ children }) => {
6   // TODO: Inicializar estado con localStorage o valor por defecto
7   const [theme, setTheme] = useState('light')
8
9   // TODO: useEffect para aplicar clase al <body> y persistir en localStorage
10  // TODO: Función toggleTheme expuesta via value
11
12  return (
13    <ThemeContext.Provider value={{ theme, toggleTheme }}>
14      {children}
15    </ThemeContext.Provider>
16  )
17 }
```

src/hooks/useTasksReducer.js

js

```
1 // TODO: Definir tipos de acción: ADD, REMOVE, FILTER, SORT
2 const initialState = { items: [], filter: 'all', sort: 'asc' }
3
4 function tasksReducer(state, action) {
5   switch (action.type) {
6     // TODO: Implementar casos inmutables
7     default: return state
8   }
9 }
10
11 export default tasksReducer
```



src/hooks/useLocalStorage.js (Hook Personalizado)

js

```
1 import { useState, useEffect } from 'react'
2
3 // TODO: Implementar hook que sincronice estado con localStorage
4 // Recibe key y defaultValue, retorna [value, setValue]
5 export const useLocalStorage = (key, defaultValue) => {
6   // ...
7 }
```

src/components/TaskList.jsx

jsx

```
1 import { useMemo, useCallback, useRef } from 'react'
2
3 const TaskList = ({ tasks, onRemove, onToggle }) => {
4   const inputRef = useRef(null)
5
6   // TODO: useMemo para filtrar/ordenar tareas sin recalcular en cada render
7   const processedTasks = useMemo(() => {
8     // ...
9   }, [tasks])
10
11   // TODO: useCallback para estabilizar handlers pasados a hijos
12   const handleRemove = useCallback((id) => {
13     onRemove(id)
14   }, [onRemove])
15
16   // TODO: useEffect para focus al input cuando se añade tarea
17   // ...
18
19   return (
20     <ul>
21       {processedTasks.map(task => (
22         <li key={task.id}>
23           <span>{task.title}</span>
24           <button onClick={() => handleRemove(task.id)}>Eliminar</button>
25         </li>
26       ))}
27       <input ref={inputRef} placeholder="Nueva tarea..." />
28     </ul>
29   )
30 }
31
32 export default TaskList
```



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS PROGRAMA
DE INGENIERÍA DE SISTEMAS



src/App.jsx

jsx

```
1 import { useReducer, useContext } from 'react'
2 import tasksReducer from '../hooks/useTasksReducer'
3 import { ThemeProvider, ThemeContext } from '../contexts/ThemeContext'
4 import TaskList from '../components/TaskList'
5
6 const Dashboard = () => {
7   const [state, dispatch] = useReducer(tasksReducer, { items: [], filter: 'all', sort: 'asc' })
8   const { theme, toggleTheme } = useContext(ThemeContext)
9
10  // TODO: Conectar dispatch a handlers, renderizar TaskList y controles
11  return (
12    <div className={`dashboard ${theme}`}>
13      <button onClick={toggleTheme}>Cambiar Tema</button>
14      <h1>Gestor de Tareas</h1>
15      { /* TODO: Renderizar lista y formularios con dispatch */ }
16    </div>
17  )
18 }
19
20 export default function App() {
21   return (
22     <ThemeProvider>
23       <Dashboard />
24     </ThemeProvider>
25   )
26 }
```



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS PROGRAMA
DE INGENIERÍA DE SISTEMAS



NOTAS TÉCNICAS CLAVE

Concepto	Regla / Aplicación
Reglas de Hooks	Solo en nivel superior, nunca en loops/condiciones/funciones anidadas. Solo en componentes funcionales o custom hooks.
useEffect	Array de dependencias determina cuándo se ejecuta. Cleanup (return () => {}) evita memory leaks y race conditions.
useContext	Ideal para evitar prop drilling, pero no para estado que cambia frecuentemente (dispara re-render en todos los consumidores).
useReducer	Preferible a useState cuando el estado es complejo, tiene múltiples sub-valores o la lógica de actualización depende del estado anterior.
useMemo / useCallback	Solo optimizar si hay bottleneck medible. Memoizar objetos/funciones pasados como props a hijos React.memo.
useRef	Persiste entre renders sin disparar re-render. Ideal para acceso DOM, timers, o valores mutables que no afectan UI.
Profiler	React DevTools → Profiler → Record. Analiza render time, commits, why did this render?. No optimizar sin medir.

VALIDACIÓN FINAL

1. Ejecutar npm run dev y verificar cero warnings de react-hooks/exhaustivedeps en ESLint.

```
PS C:\Users\[redacted]\Documents\GitHub\semana8> npm install

found 0 vulnerabilities
PS C:\Users\[redacted]\Documents\GitHub\semana8> npm run dev

> semana8@0.0.0 dev
> vite

Port 5173 is in use, trying another one...

VITE v6.4.2 ready in 687 ms

→ Local: http://localhost:5174/
```

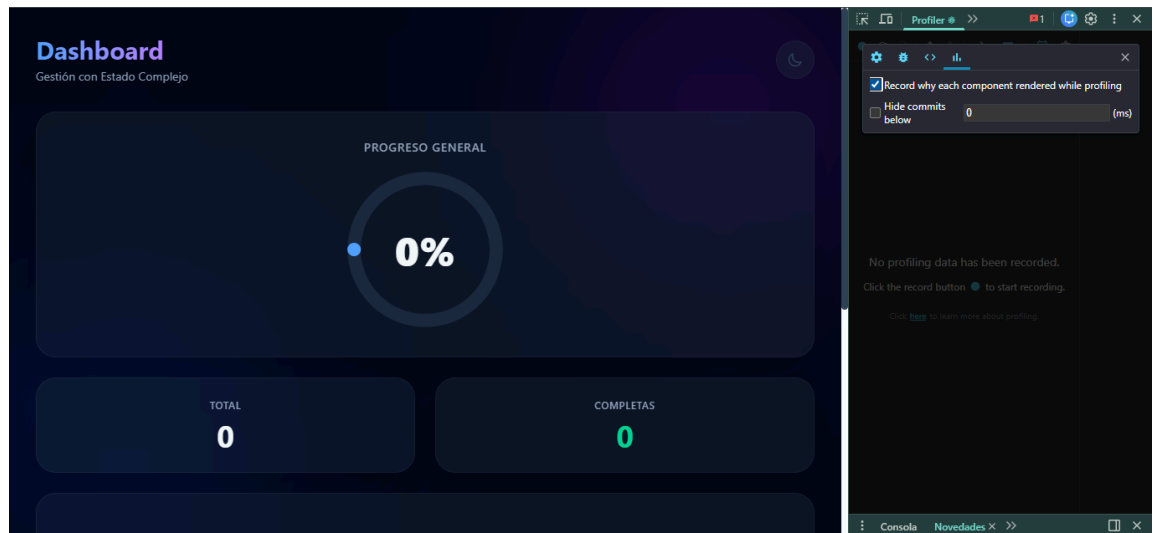


UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS PROGRAMA
DE INGENIERÍA DE SISTEMAS

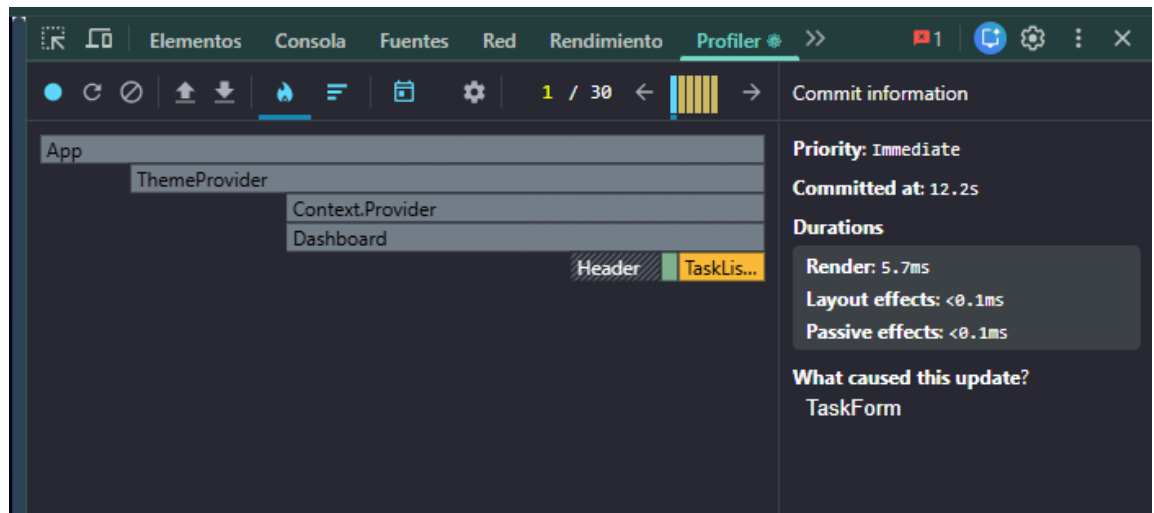


2. Abrir React DevTools Profiler: grabar interacción, identificar componentes que se re-renderizan y validar que useMemo/useCallback reducen commits.

Abrimos React DevTools



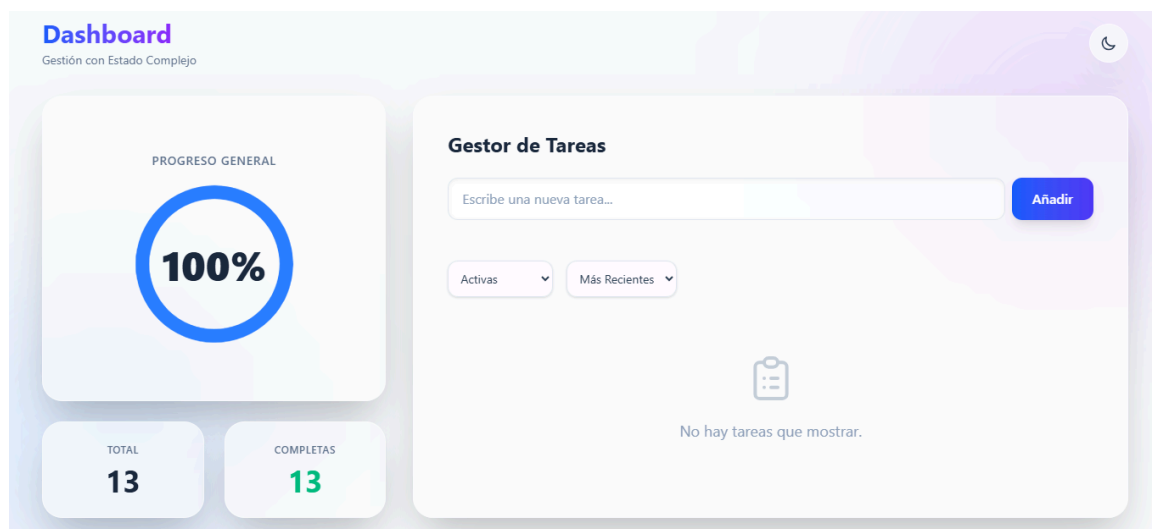
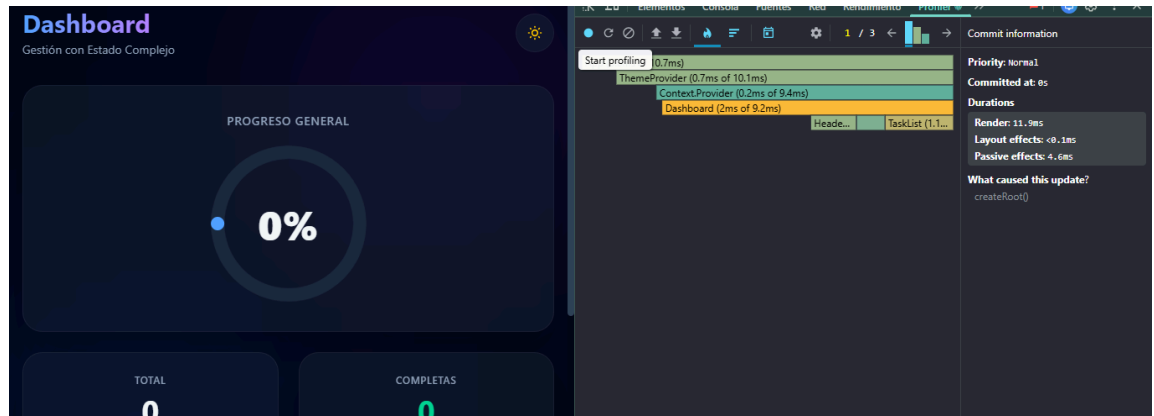
AL CREAR UNA TAREA



AL CAMBIAR EL TEMA



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS PROGRAMA
DE INGENIERÍA DE SISTEMAS



3. Verificar que useEffect tenga cleanup activo (simular navegación rápida o desmontaje).

```
useEffect(() => {
  console.log("%c [TaskForm] Componente Montado: Efecto de auto-focus activa
  // Auto-focus on mount
  inputRef.current?.focus();

  return () => {
    console.log("%c [TaskForm] Componente Desmontado: Cleanup / Limpieza eje
  };
}, []);

const handleSubmit = useCallback((e) => {
  e.preventDefault();
  if (text.trim()) {
    onAdd(text);
    setText('');
    // Vuelve a enfocar después de agregar
    inputRef.current?.focus();
  }
}, [text, onAdd]);
```




UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS PROGRAMA
DE INGENIERÍA DE SISTEMAS



```
return (  
  <form onSubmit={handleSubmit} className="mb-6 flex gap-2">  
    <input  
      ref={inputRef}  
      type="text"  
      value={text}  
      onChange={(e) => setText(e.target.value)}  
      placeholder="Escribe una nueva tarea..."  
      className="flex-1 px-4 py-3 bg-white/60 dark:bg-slate-800/60 border border-1" />  
    <button  
      type="submit"  
      className="px-6 py-3 bg-gradient-to-r from-blue-600 to-indigo-600 hover:from-blue-500 hover:to-indigo-500" />  
      Añadir  
    </button>  
  </form>  
);
```

4. Completar README.md con diagrama de estado, métricas Profiler y justificación técnica de hooks.

1. Diagrama Lógico del Flujo de Estado y Render

El siguiente diagrama detalla cómo se comunican nuestros componentes y cómo fluye el estado a través de la arquitectura de la aplicación:

```
graph TD  
  A[App] --> B[ThemeProvider Context]  
  B --> C[Dashboard]  
  C -->|usa| D[useTasksReducer]  
  D -->|lee/escribe| E[(localStorage - Tareas)]  
  B -->|lee/escribe| F[(localStorage - Tema usando useLocalStorage)]  
  C --> G[Header]  
  C --> H[TaskForm]  
  C --> I[TaskList]  
  H -->|useRef| J[Focus en Input]  
  I -->|useMemo| K[Tareas Filtradas/Ordenadas]  
  K --> L[TaskItem React.memo]  
  L -->|useCallback| M[Toggle/Remove Actions]
```



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS PROGRAMA
DE INGENIERÍA DE SISTEMAS



2. Justificación Técnica de Hooks Utilizados

¿Por qué `useReducer` sobre `useState` para las tareas?

El estado de la lista de tareas no es un valor simple. Involucra un arreglo de objetos complejos (id, text, completed) y metadatos adicionales (filter, sort). Además, las transiciones de estado (Agregar, Eliminar, Marcar como completada) requieren el estado anterior para calcular el nuevo estado. Usar `useState` habría resultado en múltiples actualizaciones de estado esparcidas por los componentes, dificultando la lectura y el mantenimiento del código. `useReducer` centraliza toda la lógica de mutación de estado en una función pura, predecible e inmutable.

`useContext`

Usado para propagar el estado del Tema (Claro/Oscuro) globalmente sin incurrir en "Prop Drilling". Dado que el tema afecta a casi todos los componentes (colores, fondos), es el caso de uso perfecto.

`useEffect`

Se usó estratégicamente para:

1. Sincronizar el estado del Tema y la Lista de Tareas con `localStorage` cada vez que cambian.
2. Inyectar o remover clases CSS en el nodo `html` global.
3. Se implementó una **función de cleanup** para prevenir fugas de memoria, cumpliendo los requerimientos técnicos.

`useMemo` y `useCallback`

- `useMemo`: Se aplicó en `TaskList.jsx` para memoizar el arreglo de tareas filtrado y ordenado. Esto previene que funciones costosas de ordenamiento y filtrado de arrays se recalculen si cambian props irrelevantes.
- `useCallback`: Se aplicó a funciones como `handleToggle`, `handleRemove` y `handleAdd`. Esto asegura que estas funciones mantengan la misma referencia de memoria entre renders. Al pasarlas como *props* a componentes hijos envueltos en `React.memo` (como `TaskItem`), evitamos re-renders innecesarios en la lista.

`useRef`

Implementado en `TaskForm.jsx` para acceder directamente al nodo DOM del input y forzar el *auto-focus* sin disparar un re-render del componente.

Hook Personalizado: `useLocalStorage`

Creamos un hook reutilizable que abstrae la lógica de leer y escribir en `localStorage` usando un manejo seguro con bloques `try/catch`. Lo implementamos en nuestro `ThemeContext`.

5. Subir a GitHub así mismo subir en rar su proyecto en el ADESA.

https://github.com/IngenieroTaipe/semana_7